GENERATORS AND OBJECTS Meta

COMPUTER SCIENCE MENTORS 61A

October 14 – October 17, 2025

Example Timeline

- Ice Breaker [5 min]

  If you want, start off with an icebreaker of your choice!

  Also, there is a link to a feedback form at the bottom of this worksheet! It would be amazing if you could highlight this/fill it out yourself so we can improve the worksheets in the future!
- Mini-lecture: Generators [5 min]
- Q1: Accumulate [5 min]
- Q2: All Sums [8 min]
- Q3: Fruit Options [8 min]
- Mini-lecture: Objects and Attributes [7 min]
- Q4: Build A Bear [7 min]
- Q5: Photosynthesis [7 min]
- Q6: Musician [extra time]

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28

1 Generators

1. Write a generator function that takes in an iterator `it` and yields the running total of the elements produced by `it`.

```
def accumulate(it):
    '''
    >>> def all_ints():
    ...     i = 0
    ...     while True:
    ...         yield i
    ...         i += 1
    >>> a = accumulate(all_ints())
    >>> [next(a) for x in range(6)]
    [0, 1, 3, 6, 10, 15]
    '''
```

```
def accumulate(it):
    sum = 0
    while True:
        sum += next(it)
        yield sum
```

This problem is meant to be a relatively simple introduction to the manipulation of iterators, generator function syntax, and how to deal with infinite generators.

The doc test for this question is deliberately a bit complicated. It actually serves as a bit of a hint for how we might construct an infinite generator if one is provided to us (using an infinite loop). If students are concerned about this possibly erroring, remind them that generators only execute the body of the function when we tell them to, so this will not actually cause an infinite loop that hogs all of our computer's resources, causing a crash.

Students have a tendency to want to do everything in generators with lists, which requires the computation of all elements of the generator at the same time; we need to encourage them to do "lazy" iteration, which means that each element is only computed when it's needed. Lazy iteration is a core concept of the topic; not only is it more efficient to do things this way, it's literally the only way to handle infinite iterators (whose elements cannot all be computed at the same time). Whenever I do iteration problems with my students, I ask them to consider what would happen if we provided them an infinite generator; if their implementation breaks, then it's not correct.

Suggested Time: 5 min

2. Define `all_sums`, a generator that iterates through all the possible sums of elements from `lst`. (Repeat sums are permitted.)

```
def all_sums(lst):
    '''
    >>> list(all_sums([]))
    [0]
    >>> list(all_sums([1, 2]))
    [3, 2, 1, 0]
    >>> list(all_sums([1, 2, 3]))
    [6, 5, 4, 3, 3, 2, 1, 0]
    >>> list(all_sums([1, 2, 7]))
    [10, 9, 8, 7, 3, 2, 1, 0]
    '''


    if len(lst) == 0:
        yield 0
    else:
        for sum_rest in all_sums(lst[1:]):
            yield sum_rest + lst[0]
            yield sum_rest
```

Teaching Tips

- This is a classic tree recursion problem but now in generator form!
- A tree diagram of how the list splits is a good visualization to draw
- Students may have trouble with this because the order in which they're dealing with the recursive case is a bit different than usual.
- If students are struggling to understand the problem, start from the base case of an empty list and work your way up with the sums of a length-1 list, length-2, etc.
- As always, the recursive leap of faith is helpful in understanding what `all_sums(lst[1:])` returns.
- Even though this is a generator problem, we iterate over the call in a for loop so we can treat the function like it returns a list!

Suggested Time: 8 min

3. Write a generator that, given m (the amount of money you have), pc (the cost of one pear), and ac (the cost of one apple) yields all possible combinations of fruit that you can buy that uses up the most of your money. In other words, each combination of fruits should not result in enough money left over to buy another fruit. Combinations of the same number of each fruit in different orders is okay. It is also okay if each combination has an extra space at the end.

```
def fruitOptions(m, pc, ac):
    '''
    >>> print(list(fruitOptions(10, 2, 5)))
    ['pear pear pear pear pear ', 'pear pear apple ', 'pear apple pear
       ',
     'apple pear pear ', 'apple apple ']
    '''
    if ____________________________________:
        yield ''
    if m >= pc:
        for ____________________________________________:
            ____________________________________
    if m >= ac:
        for ____________________________________________:
            ____________________________________________


    def fruitOptions(m, pc, ac):
        if m < pc and m < ac:
            yield ''
        if m >= pc:
            for p in fruitOptions(m-pc, pc, ac):
                yield 'pear ' + p;
        if m >= ac:
            for a in fruitOptions(m-ac, pc, ac):
                yield 'apple ' + a;
```

Teaching Tips

- This problem shows that some generator problems can be approached in a similar way as recursion problems, except with yield instead of return.
- Another goal of this problem is to show how iterating through a generator can be done through a through a $for\_\_in\_\_$ loop.

Suggested Time: 8 min Feel free to also skip and do the extra oop question instead if it's a later session in the week.

2 Objects and Attributes

1. Let's build a Bear using OOP!

   Bear instances should have an attribute `name` that holds the name of the bear. The Bear class should have an attribute `bears`, a list that stores the name of each bear.

```
>>> oski = Bear('Oski')
>>> oski.name
'Oski'
>>> Bear.bears
['Oski']
>>> winnie = Bear('Winnie')
>>> Bear.bears
['Oski', 'Winnie']
```

```
class Bear:

    bears = []
    def __init__(self, name):
        self.name = name
        Bear.bears.append(self.name)
```

Note that `self.bears.append(self.name)` also works, but just doing `bears.append(self.name)` will result in an error!
There is no `bears` variable in the `__init__` function frame.

Suggested Time: 7 min

2. Implement the classes so the following code runs.

```
'''
>>> p = Plant()
>>> p.height
1
>>> p.materials
[]
>>> p.absorb()
>>> p.materials
[|Sugar|]
>>> Sugar.sugars_created
1
>>> p.leaf.sugars_used
0
>>> p.grow()
>>> p.materials
[]
>>> p.height
2
>>> p.leaf.sugars_used
1
'''


class Plant:
    def __init__(self):
        self.leaf = Leaf(self)
        self.materials = []
        self.height = 1

    def absorb(self):
        self.leaf.absorb()

    def grow(self):
        for sugar in self.materials:
            sugar.activate()
            self.height += 1

class Leaf:
    def __init__(self, plant): # plant is a Plant instance
        self.alive = True
        self.sugars_used = 0
        self.plant = plant

    def absorb(self):
        if self.alive:
            self.plant.materials.append(Sugar(self, self.plant))


    def __repr__(self):
        return '|Leaf|'
```

```
class Sugar:
    sugars_created = 0

    def __init__(self, leaf, plant):
        self.leaf = leaf
        self.plant = plant
        Sugar.sugars_created += 1

    def activate(self):
        self.leaf.sugars_used += 1
        self.plant.materials.remove(self)

    def __repr__(self):
        return '|Sugar|'
```

Suggested Time: 8 min

3. **Musician** What would Python display? Write the result of executing the code and the prompts below. If a function is returned, write 'Function'. If nothing is returned, write 'Nothing'. If an error occurs, write 'Error'.

```
class Musician:
    popularity = 0
    def __init__(self, instrument):
        self.instrument = instrument
    def perform(self):
        print('a stellar ' + self.instrument + ' performance')
        self.popularity = self.popularity + 2
    def __repr__(self):
        return self.instrument

class BandLeader(Musician):
    def __init__(self):
        self.band = []
    def recruit(self, musician):
        self.band.append(musician)
    def perform(self, song):
        for m in self.band:
            m.perform()
        Musician.popularity += 1
        print(song)
    def __str__(self):
        return 'Here's the band!'
    def __repr__(self):
        band = ''
        for m in self.band:
            band += str(m) + ' '
        return band[:-1]

miles = Musician('trumpet')
goodman = Musician('clarinet')
ellington = BandLeader()
```

Some Quick Refreshers

Defining attributes: Instance attributes are defined with the `self.attr_name` notation (usually in `__init__` but could be elsewhere like in this problem). Class attributes are defined outside of methods in the body of the class definition, like the variable `popularity` in the class `Musician`.

Accessing attributes: Instance attributes are referred to using `self.attr_name` Class attributes can be referred to using `classname.attr_name` or `self.attr_name` (Note: using the latter will only work if there are no instance attributes bound with the name `attr_name`).

Before running any of the code below, `miles` and `goodman` are set to the musicians created as a result of calling the `__init__` constructor method in `Musician`. `ellington` uses `BandLeader's __init__` method, since `BandLeader` is the subclass and has `__init__` defined.

```
>>> ellington.recruit(goodman)
>>> ellington.perform()
```

Error

`ellington.recruit(goodman)` adds `goodman` to the end of `ellington's` instance attribute, `band`. Then, `ellington` checks its class (`BandLeader`) for the `perform()` method. But this `perform()` is expecting an argument, so this errors.

```
>>> ellington.perform('sing, sing, sing')
```

a rousing clarinet performance
sing, sing, sing

Using the same `perform()` method, now providing the correct number of arguments. First, going through the band list, `goodman` calls its `perform()` method, which is defined in `Musician`. Here, we print `'a rousing' + goodman`'s instrument `+ ' performance'`, and then `goodman`'s `self.popularity = self.popularity + 2` happens. The `self.popularity` on the right of the equal sign is `Musician.popularity` because `goodman` doesn't have its own instance attribute named `popularity` yet; then it becomes `self.popularity = 0 + 2`, and this creates the instance attribute `popularity` for `goodman`. Then `Musician.popularity`, the class attribute, in incremented by 1.

```
>>> goodman.popularity, miles.popularity
```

(2, 1)
First, we try to get the value of `goodman.popularity`. In our environment diagram, we see that `goodman` has the instance variable `popularity` already defined. Therefore, we get that value, 2, back. Then, we try to access `miles.popularity`. In this case, `miles` doesn't have a `popularity` instance variable defined, so we default to the class variable. There, we see it defined as 1, so we get that value. Finally, since commas in Python define a tuple, we return the two values as `(2, 1)`.

```
>>> ellington.recruit(miles)
>>> ellington.perform('caravan')
```

a rousing clarinet performance
a rousing trumpet performance
caravan
First, we call `ellington.recruit(miles)`. This appends `miles` to `ellington`'s instance variable, `band`. After that, we call `ellington.perform('caravan')`. Similar to the previous call on perform, we will loop through all of the values in `ellington.band`, calling their perform methods in order. This causes the first two lines to be printed. Next, we increment `Musician.popularity` (the class variable of `Musician` called `popularity`). Lastly, we print the `song` variable that was passed in, completing the last line.

```
>>> ellington.popularity, goodman.popularity, miles.popularity
```

(2, 4, 3)

```
>>> print(ellington)
```

Here's the band!
`print``()` expects the string representation of `ellington`, which is given by calling the `__str__()` method of `ellington`. `ellington` checks to see if `BandLeader` has a `__str__()` method, which it does. So, **`print`**`(ellington)` then becomes **`print`**`('Here's the band!')`.

```
>>> ellington
```

clarinet trumpet
When prompting for `ellington`'s value, we return the representation of ellington given by `__repr__()`. So, we call `BandLeader`'s `__repr__()` method.

Teaching Tips

- For the error, it's important to make sure your students realize that BandLeader overrides Musician's `perform` function, and therefore a function call without the correct number of parameters in the new function will not work.
- Clarify to students the difference between `__str__` and `__repr__`, especially that **`print`** implic-

itly calls `__str__` and `__repr__`.

  - Another nuance to this which may confuse students is the `__repr__` method in BandLeader, which calls **`str`** on all Musicians in the band. Since the Musician class does not have a defined `__str__` method, it defaults to the defined `__repr__` method.

- The main challenge with this problem is distinctly identifying the class and instance variables and modifying both separately.
  - In particular, every Musician begins with a class variable `popularity`. However, after the first call to `perform`, a new instance variable `self.popularity` is created, which begins with the value `Musician.popularity` + 2.
  - After this first call to a Musician's `perform`, successive calls will increment the respective instance variable by 2.
  - Calling a BandLeader's `perform` function will increment the class variable `Musician.popularity`, which will raise the starting popularities of any new Musicians after their initial performances.
  - Ensure that students understand the interplay between the `popularity` class and instance variable.

Extra Time